Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

AI Vibe Coding for Python Applications

Course Code: C179

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	22 July 2026	Initial release of course C179 — 3 days, 5 topics and 20 hands-on labs building the CardGuard fraud-screening application with an AI coding assistant. Day 3 closes with an end-to-end consolidation build and an extended course-wide recap.	Dr. Alfred Ang

TABLE OF CONTENTS

Introduction

This guide accompanies AI Vibe Coding for Python Applications (C179), a three-day hands-on course in which you build and deploy one real Python application — CardGuard, a card-transaction fraud screening system — using an AI coding assistant throughout.

The twenty labs are cumulative: each one extends the same application, so by the end you have a working three-tier system you built yourself. Every lab finishes with a verification step, because the central discipline of vibe coding is proving that the generated code actually does what you specified.

Course Learning Outcomes

- LO1: Apply vibe coding practices — set up a reproducible Python project with uv, and use an AI coding assistant to scaffold, explain and refactor working code.
- LO2: Design and implement object-oriented Python — classes, encapsulation, inheritance and composition — using AI-assisted conversational design.
- LO3: Build data analytics pipelines with pandas — load, clean, transform, group and aggregate realistic datasets into reportable results.
- LO4: Model and validate data with Pydantic and expose it through a FastAPI application with typed request and response models.
- LO5: Package and deploy a Python application — dependency locking, configuration, containerisation and a running deployed service.

Before You Start — Preparation

What you need

- A laptop with administrative rights to install software.
- Python 3.12 and uv (the labs install uv in Lab 1).
- An AI coding assistant — Claude, GitHub Copilot or Cursor.
- A terminal and a code editor (VS Code recommended).

Verify your setup

Confirm your setup before you begin Lab 1.

```
$ uv --version
```

```
$ python3 --version
```

Conventions used in every lab

- Placeholders such as <VALUE> are replaced with your own values.
- Commands prefixed with $ are typed into your terminal.
- Every lab ends with a 'Test it' step — do not skip it.

Topic 01 — Vibe Coding Foundations

uv Projects · AI Coding Assistants · Prompting Patterns · AI-Assisted Refactoring

Key concepts

- Vibe coding is directing an AI assistant to write code you specify, review and own — you remain accountable for correctness.
- uv manages the interpreter, virtual environment and locked dependencies: uv init, uv add, uv run.
- Effective prompts state the goal, the constraints, the inputs and the expected output — vague prompts produce plausible but wrong code.
- Always read and run AI-generated code before trusting it; refactoring conversationally is faster than rewriting by hand.

Lab 1 — Set Up a Reproducible Python Project with uv

Learning outcome: Create and run a Python project using uv.

Goal: The learner installs uv, creates a project, adds locked dependencies, and runs code through uv so the environment is identical on every machine. This is the foundation every later lab builds on.

What you'll build

A uv project with pyproject.toml, uv.lock and a working entry point (Tools: uv, Python 3.12, pyproject.toml, uv.lock.)

Step-by-step

1. Confirm uv is installed and check the version

```
uv --version
```

2. Create a new project folder and initialise it

```
uv init cardguard
```

```
cd cardguard
```

3. Inspect what uv generated — note pyproject.toml is the project manifest

```
ls -a
```

```
cat pyproject.toml
```

4. Pin the Python version so every learner runs the same interpreter

```
uv python pin 3.12
```

5. Add a dependency — uv resolves, installs and writes uv.lock in one step

```
uv add pandas
```

6. Open uv.lock and observe the exact pinned versions

```
head -30 uv.lock
```

7. Write a first script that proves the dependency is importable

```
# main.py

import pandas as pd


def main() -> None:

    df = pd.DataFrame({"card": ["4111...1111", "5500...0004",
"4111...1111"],

                       "merchant": ["SG Grocer", "Overseas ATM", "SG
Grocer"],

                       "amount": [42.90, 800.00, 15.50]})

    print(df)

    print(f"Total screened: ${df['amount'].sum():,.2f}")


if __name__ == "__main__":

    main()
```

8. Run it through uv — no venv activation needed

```
uv run python main.py
```

9. Prove reproducibility: delete the venv and restore it from the lockfile

```
rm -rf .venv

uv sync

uv run python main.py
```

Test it

uv run python main.py prints the transaction DataFrame and the screened total. After deleting .venv, uv sync restores it and the script produces identical output.

Note: Full commands and screenshots are in labs/lab-01-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 2 — Write Effective Prompts for an AI Coding Assistant

Learning outcome: Apply prompting patterns that produce correct, reviewable code.

Goal: The learner contrasts a vague prompt with a specified prompt on the same task, then applies the goal-constraints-inputs-output pattern to generate a function that handles real edge cases.

What you'll build

Two versions of a transaction risk-scoring function and a written comparison of the prompts that produced them (Tools: AI coding assistant (Claude / Copilot / Cursor), uv, Python.)

Step-by-step

1. Start from the vague prompt and record exactly what you get back

```
Ask the assistant: write a function to score a transaction
```

2. Read the generated code critically — list what it assumed. Typical gaps: what counts as high value, whether an overseas merchant matters, what the score range is, what happens for a negative amount.
3. Rewrite the prompt using the four-part pattern — goal, constraints, inputs, expected output

```
Ask the assistant: Write a Python function score_transaction(amount: float,
is_overseas: bool, hour: int) -> dict that returns a risk score from 0 to
100 and the reasons that contributed to it. Add 40 points when the amount
exceeds $500, 30 points when the merchant is overseas, and 20 points when
the hour is between 1am and 5am. Cap the score at 100. Raise ValueError for
a negative amount or an hour outside 0-23. Return the score and a list of
reason strings.
```

4. Save the improved result and read every line before running it

```
# scoring.py

def score_transaction(amount: float, is_overseas: bool, hour: int) -> dict:

    """Return a 0-100 risk score for one transaction, with its reasons."""

    if amount < 0:

        raise ValueError("amount must not be negative")

    if not 0 <= hour <= 23:

        raise ValueError("hour must be between 0 and 23")


    HIGH_VALUE = 500.00
```

```
    score, reasons = 0, []

    if amount > HIGH_VALUE:
        score += 40
        reasons.append("high value")
    if is_overseas:
        score += 30
        reasons.append("overseas merchant")
    if 1 <= hour <= 5:
        score += 20
        reasons.append("unusual hour")

    return {"score": min(score, 100), "reasons": reasons}
```

5. Test the boundaries the prompt specified — this is where generated code usually fails

```
uv run python -c "
from scoring import score_transaction
print(score_transaction(42.90, False, 14))   # low risk
print(score_transaction(800.00, True, 3))    # every rule fires
print(score_transaction(500.00, False, 14))  # exactly on the threshold
"
```

6. Confirm the error paths actually raise

```
uv run python -c "
from scoring import score_transaction
try:
    score_transaction(-100, False, 14)
except ValueError as e:
    print('correctly raised:', e)
"
```

7. Write two sentences in your notes: which prompt produced code you would ship, and why.

Test it

The specified prompt produces a function that caps the score at 100, fires each rule independently, and raises ValueError on invalid input. The learner can state why the vague prompt was insufficient.

Note: Full commands and screenshots are in labs/lab-02-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 3 — Review and Correct AI-Generated Code

Learning outcome: Identify and fix defects in code produced by an AI assistant.

Goal: The learner is given a plausible but subtly wrong implementation, finds the defects by testing rather than by reading alone, and corrects them. This lab establishes that the developer, not the assistant, owns correctness.

What you'll build

A corrected discount calculator plus a short defect log (Tools: AI coding assistant, uv, Python.)

Step-by-step

1. Save this generated function exactly as written — it looks reasonable

```
# discount.py

def apply_discount(price, tier):

    if tier == "gold":

        return price * 0.8

    elif tier == "silver":

        return price * 0.9

    elif tier == "bronze":

        return price * 0.95

    return price
```

2. Probe it with the values a real order system would send

```
uv run python -c "

from discount import apply_discount
```

```
print(apply_discount(100, 'gold'))

print(apply_discount(100, 'GOLD'))

print(apply_discount(100, None))

print(apply_discount(-50, 'gold'))

print(apply_discount(100.555, 'silver'))

"
```

3. Record the defects you found: case sensitivity, no validation of price, unrounded currency, silent fallthrough on an unknown tier, and no type hints.

4. Ask the assistant to fix the specific defects — name them, do not say 'improve this'

```
Ask the assistant: Fix these defects in apply_discount: (1) tier matching
must be case-insensitive, (2) raise ValueError for a negative price, (3)
raise ValueError for an unrecognised tier instead of silently returning the
full price, (4) round the result to 2 decimal places, (5) add type hints
and a docstring.
```

5. Review the corrected version line by line before accepting it

```
# discount.py

DISCOUNTS = {"gold": 0.20, "silver": 0.10, "bronze": 0.05}


def apply_discount(price: float, tier: str) -> float:

    """Return the price after applying the tier discount."""

    if price < 0:

        raise ValueError("price must not be negative")

    key = (tier or "").strip().lower()

    if key not in DISCOUNTS:

        raise ValueError(f"unknown tier: {tier!r}")

    return round(price * (1 - DISCOUNTS[key]), 2)
```

6. Re-run every probe that previously failed

```
uv run python -c "

from discount import apply_discount

print(apply_discount(100, 'GOLD'))

print(apply_discount(100.555, 'silver'))
```

```
for bad in [(-50, 'gold'), (100, 'platinum'), (100, None)]:
    try:
        apply_discount(*bad)
        print('NOT CAUGHT:', bad)
    except ValueError as e:
        print('correctly raised:', e)
"
```

7. Note the lesson: the first version ran without error on the happy path. Running is not the same as correct.

Test it

All five defects are fixed and demonstrated by tests. The learner can explain why reading alone did not reveal them.

Note: Full commands and screenshots are in labs/lab-03-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 4 — Refactor Working Code Conversationally

Learning outcome: Use an AI assistant to restructure code without changing its behaviour.

Goal: The learner takes a working but poorly structured script, establishes a behavioural baseline, refactors it into small testable functions with AI assistance, and proves the output is unchanged.

What you'll build

A refactored sales summary script with an unchanged, verified output (Tools: AI coding assistant, uv, Python.)

Step-by-step

1. Save the working script — it produces correct output but is one long block

```
# sales_report.py
orders = [
    {"id": 1, "region": "North", "amount": 1200.50, "status": "paid"},
    {"id": 2, "region": "South", "amount": 890.00, "status": "paid"},
```

```
    {"id": 3, "region": "North", "amount": 450.25, "status": "refunded"},
    {"id": 4, "region": "East",  "amount": 2100.75, "status": "paid"},
    {"id": 5, "region": "South", "amount": 310.00, "status": "pending"},
]


totals = {}
for o in orders:
    if o["status"] == "paid":
        if o["region"] not in totals:
            totals[o["region"]] = 0
        totals[o["region"]] += o["amount"]
for r in sorted(totals):
    print(f"{r}: ${totals[r]:,.2f}")
print(f"TOTAL: ${sum(totals.values()):,.2f}")
```

2. Capture the baseline output to a file — this is the contract the refactor must preserve

```
uv run python sales_report.py > baseline.txt
cat baseline.txt
```

3. Ask for a structural refactor and state explicitly that behaviour must not change

```
Ask the assistant: Refactor sales_report.py into three functions —
filter_paid(orders), total_by_region(orders) and format_report(totals) —
plus a main() guarded by if __name__ == '__main__'. Add type hints. The
printed output must be byte-for-byte identical to the current version.
```

4. Save the refactored version

```
# sales_report.py
from typing import Iterable


Order = dict[str, object]


ORDERS: list[Order] = [
    {"id": 1, "region": "North", "amount": 1200.50, "status": "paid"},
```

```
    {"id": 2, "region": "South", "amount": 890.00, "status": "paid"},
    {"id": 3, "region": "North", "amount": 450.25, "status": "refunded"},
    {"id": 4, "region": "East",  "amount": 2100.75, "status": "paid"},
    {"id": 5, "region": "South", "amount": 310.00, "status": "pending"},
]

def filter_paid(orders: Iterable[Order]) -> list[Order]:
    """Return only the orders with status 'paid'."""
    return [o for o in orders if o["status"] == "paid"]

def total_by_region(orders: Iterable[Order]) -> dict[str, float]:
    """Sum order amounts grouped by region."""
    totals: dict[str, float] = {}
    for o in orders:
        region = str(o["region"])
        totals[region] = totals.get(region, 0.0) + float(o["amount"])
    return totals

def format_report(totals: dict[str, float]) -> str:
    """Render the per-region totals and the grand total."""
    lines = [f"{r}: ${totals[r]:,.2f}" for r in sorted(totals)]
    lines.append(f"TOTAL: ${sum(totals.values()):,.2f}")
    return "\n".join(lines)

def main() -> None:
    print(format_report(total_by_region(filter_paid(ORDERS))))

if __name__ == "__main__":
```

```
    main()
```

5. Prove the behaviour is unchanged by diffing against the baseline

```
uv run python sales_report.py > after.txt

diff baseline.txt after.txt && echo 'IDENTICAL — refactor is safe'
```

6. Now that the logic is in functions, test one piece in isolation — impossible before the refactor

```
uv run python -c "

from sales_report import total_by_region

rows = [{'region': 'North', 'amount': 100.0, 'status': 'paid'},

        {'region': 'North', 'amount': 50.0,  'status': 'paid'}]

assert total_by_region(rows) == {'North': 150.0}

print('unit test passed')

"
```

7. Note the sequence that makes refactoring safe: baseline first, refactor second, diff third.

Test it

diff reports no difference between baseline.txt and after.txt, and total_by_region can be unit-tested independently.

Note: Full commands and screenshots are in labs/lab-04-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Topic 02 — Object-Oriented Programming in Python

Classes · Encapsulation · Inheritance · Composition · Dunder Methods

Key concepts

- A class bundles data (attributes) with the behaviour that operates on it (methods); an object is one instance of it.
- Encapsulation hides internal state behind methods and properties so callers depend on behaviour, not layout.
- Inheritance models 'is-a' and shares behaviour; composition models 'has-a' and is usually the more flexible default.

- Dunder methods (__init__, __repr__, __eq__) integrate your classes with Python's built-in operators and printing.

Lab 5 — Model a Domain with Classes and Dunder Methods

Learning outcome: Define classes that bundle data with behaviour.

Goal: The learner replaces loose dictionaries with a Transaction class, adds __init__, __repr__ and __eq__, and sees why a class is easier to reason about than a dict once behaviour is attached to the data.

What you'll build

A Transaction class with construction, printing, equality and derived properties (Tools: uv, Python 3.12, dataclasses, sqlite3.)

Step-by-step

1. Start the project and generate the database

```
uv init cardguard

cd cardguard

uv add pandas

cp ../mockdata.py .

uv run python mockdata.py
```

2. Look at the raw shape of the data you are about to model

```
uv run python -c "

import sqlite3

con = sqlite3.connect('cardguard.db')

row = con.execute('SELECT * FROM transactions LIMIT 1').fetchone()

print(row)

"
```

3. Write the Transaction class — data and the behaviour that belongs with it

```
# models.py

from dataclasses import dataclass

from datetime import datetime


@dataclass
```

```
class Transaction:
    """One card transaction presented for screening."""
    id: int
    card_ref: str
    ts: datetime
    amount: float
    merchant_category: str
    city: str
    lat: float
    lon: float

    @property
    def hour(self) -> int:
        """Hour of day, 0-23 — used by the unusual-hour rule."""
        return self.ts.hour

    @property
    def is_high_risk_category(self) -> bool:
        return self.merchant_category in {"crypto_exchange", "gift_cards",
"jewellery"}

    def __repr__(self) -> str:
        return (f"Transaction(id={self.id}, card={self.card_ref}, "
                f"${self.amount:,.2f}, {self.merchant_category},
{self.city})")
```

4. Add a constructor that builds a Transaction from a database row

```
    @classmethod
    def from_row(cls, row: tuple) -> "Transaction":
        """Build a Transaction from a sqlite row tuple."""
```

```
        return cls(id=row[0], card_ref=row[1],
ts=datetime.fromisoformat(row[2]),
                   amount=row[3], merchant_category=row[4], city=row[5],
                   lat=row[6], lon=row[7])
```

5. Load real rows through the class and print them

```
uv run python -c "
import sqlite3
from models import Transaction
con = sqlite3.connect('cardguard.db')
rows = con.execute('SELECT * FROM transactions LIMIT 5').fetchall()
for r in rows:
    t = Transaction.from_row(r)
    print(t, '| hour', t.hour, '| high risk', t.is_high_risk_category)
"
```

6. Verify @dataclass gave you equality for free — dicts compare by content, objects normally by identity

```
uv run python -c "
from datetime import datetime
from models import Transaction
a = Transaction(1, 'CH0001', datetime(2026,4,1,9), 50.0, 'grocery',
'Singapore', 1.35, 103.8)
b = Transaction(1, 'CH0001', datetime(2026,4,1,9), 50.0, 'grocery',
'Singapore', 1.35, 103.8)
print('equal:', a == b)
print('same object:', a is b)
"
```

7. Note the gain: hour and is_high_risk_category live WITH the data. With dicts, that logic would be scattered across every caller.

Test it

Transaction.from_row builds objects from the database, __repr__ prints readably, __eq__ compares by value, and the derived properties return correct results.

Note: Full commands and screenshots are in labs/lab-05-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 6 — Encapsulate State with Properties and Validation

Learning outcome: Protect object state behind a controlled interface.

Goal: The learner builds a Cardholder class whose running spend baseline cannot be corrupted from outside, using private attributes, a read-only property and validation in the setter.

What you'll build

A Cardholder class with a protected baseline and a validated update method (Tools: uv, Python 3.12.)

Step-by-step

1. Write the class with state deliberately kept private

```
# cardholder.py

class Cardholder:

    """A card account with a running spend baseline used for anomaly
scoring."""


    def __init__(self, card_ref: str, name: str, home_city: str,
typical_amount: float):

        if typical_amount <= 0:

            raise ValueError("typical_amount must be positive")

        self.card_ref = card_ref

        self.name = name

        self.home_city = home_city

        self._typical_amount = typical_amount   # leading _ = internal

        self._observed_count = 0


    @property

    def typical_amount(self) -> float:
```

```
        """Read-only view of the baseline — callers cannot assign to it."""
        return round(self._typical_amount, 2)


    @property
    def observed_count(self) -> int:
        return self._observed_count
```

2. Add the only sanctioned way to move the baseline

```
    def observe(self, amount: float) -> None:
        """Fold one new transaction into the running baseline."""
        if amount < 0:
            raise ValueError("amount must not be negative")
        self._observed_count += 1
        weight = 1 / min(self._observed_count, 30)   # rolling, recency-
weighted
        self._typical_amount = (1 - weight) * self._typical_amount + weight
* amount


    def deviation_factor(self, amount: float) -> float:
        """How many times the baseline this amount represents."""
        return round(amount / self._typical_amount, 2)
```

3. Prove the baseline cannot be overwritten from outside

```
uv run python -c "
from cardholder import Cardholder
ch = Cardholder('CH0001', 'Aisha Tan', 'Singapore', 120.0)
try:
    ch.typical_amount = 999999
except AttributeError as e:
    print('blocked:', e)
"
```

4. Drive the baseline the sanctioned way and watch it move

```
uv run python -c "

from cardholder import Cardholder

ch = Cardholder('CH0001', 'Aisha Tan', 'Singapore', 120.0)

for amt in [110, 130, 125, 118, 122]:

    ch.observe(amt)

print('baseline', ch.typical_amount, 'after', ch.observed_count,
'observations')

print('a $2,400 charge is', ch.deviation_factor(2400), 'x baseline')

"
```

5. Confirm validation rejects bad input at both entry points

```
uv run python -c "

from cardholder import Cardholder

for bad in [lambda: Cardholder('C','n','SG',0), lambda:
Cardholder('C','n','SG',120).observe(-5)]:

    try:

        bad(); print('NOT CAUGHT')

    except ValueError as e:

        print('correctly raised:', e)

"
```

6. Load the real cardholders and rank them by baseline

```
uv run python -c "

import sqlite3

from cardholder import Cardholder

con = sqlite3.connect('cardguard.db')

rows = con.execute('SELECT card_ref,name,home_city,typical_amount FROM
cardholders').fetchall()

chs = [Cardholder(*r) for r in rows]

top = sorted(chs, key=lambda c: c.typical_amount, reverse=True)[:5]

for c in top:

    print(f'{c.card_ref} {c.name:<16} baseline ${c.typical_amount:,.2f}')
```

"

7. Note why this matters for fraud: if any caller could assign typical_amount directly, an attacker-controlled input path could raise the baseline until nothing looks anomalous.

Test it

Assigning to typical_amount raises AttributeError; observe() updates the baseline; both constructors and observe() reject invalid values; the 5 highest baselines print from the database.

Note: Full commands and screenshots are in labs/lab-06-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 7 — Build a Rule Hierarchy with Inheritance and Polymorphism

Learning outcome: Share behaviour through a base class and override it per rule.

Goal: The learner defines an abstract FraudRule base class and implements four concrete rules. Each rule scores a transaction differently but presents an identical interface, so the caller never needs to know which rule it holds.

What you'll build

An abstract FraudRule plus AmountDeviationRule, UnusualHourRule, HighRiskCategoryRule and VelocityRule (Tools: uv, Python 3.12, abc.)

Step-by-step

1. Define the contract every rule must satisfy

```
# rules.py

from abc import ABC, abstractmethod

from dataclasses import dataclass

from models import Transaction

from cardholder import Cardholder


@dataclass

class RuleResult:

    """One rule's verdict on one transaction."""

    rule_name: str
```

```
    score: float          # 0.0 (clean) .. 1.0 (certain)
    reason: str


class FraudRule(ABC):
    """Base class: every rule scores a transaction from 0.0 to 1.0."""

    weight: float = 1.0

    @property
    def name(self) -> str:
        return self.__class__.__name__

    @abstractmethod
    def score(self, txn: Transaction, holder: Cardholder,
              history: list[Transaction]) -> RuleResult:
        """Return this rule's verdict. Must be implemented by every
subclass."""
```

2. Prove the abstract class cannot be instantiated — the contract is enforced

```
uv run python -c "
from rules import FraudRule
try:
    FraudRule()
except TypeError as e:
    print('correctly blocked:', e)
"
```

3. Implement the amount-deviation rule

```
class AmountDeviationRule(FraudRule):
    """Flags spend far above this cardholder's own baseline."""
    weight = 1.2
```

```
    def __init__(self, alert_factor: float = 8.0):
        self.alert_factor = alert_factor

    def score(self, txn, holder, history):
        factor = holder.deviation_factor(txn.amount)
        s = min(factor / self.alert_factor, 1.0) if factor > 1 else 0.0
        return RuleResult(self.name, round(s, 3),
                          f"${txn.amount:,.2f} is {factor}x the $
{holder.typical_amount:,.2f} baseline")
```

4. Implement the unusual-hour and high-risk-category rules

```
class UnusualHourRule(FraudRule):
    """Flags activity outside the cardholder's normal waking window."""
    weight = 0.6

    def score(self, txn, holder, history):
        s = 0.8 if txn.hour in {0, 1, 2, 3, 4, 5} else 0.0
        return RuleResult(self.name, s, f"transaction at
{txn.hour:02d}:00")


class HighRiskCategoryRule(FraudRule):
    """Flags merchant categories with elevated chargeback rates."""
    weight = 0.8

    RISK = {"crypto_exchange": 0.9, "gift_cards": 0.8,
            "jewellery": 0.6, "online_gaming": 0.5}

    def score(self, txn, holder, history):
        s = self.RISK.get(txn.merchant_category, 0.0)
```

```
        return RuleResult(self.name, s, f"category
{txn.merchant_category}")
```

5. Implement the velocity rule — it needs history, which the shared interface already provides

```
from datetime import timedelta


class VelocityRule(FraudRule):

    """Flags many transactions inside a short window."""

    weight = 1.5


    def __init__(self, window_minutes: int = 10, threshold: int = 4):

        self.window = timedelta(minutes=window_minutes)

        self.threshold = threshold


    def score(self, txn, holder, history):

        recent = [h for h in history

                  if 0 <= (txn.ts - h.ts).total_seconds() <=
self.window.total_seconds()]

        n = len(recent) + 1

        s = min((n - 1) / self.threshold, 1.0) if n > 1 else 0.0

        return RuleResult(self.name, round(s, 3),

                          f"{n} transactions in {self.window.seconds // 60}
minutes")
```

6. Call every rule through the SAME interface — this is polymorphism doing real work

```
uv run python -c "

import sqlite3

from models import Transaction

from cardholder import Cardholder

from rules import AmountDeviationRule, UnusualHourRule,
HighRiskCategoryRule, VelocityRule
```

```
con = sqlite3.connect('cardguard.db')

row = con.execute('SELECT * FROM transactions WHERE is_known_fraud=1 ORDER
BY amount DESC LIMIT 1').fetchone()

txn = Transaction.from_row(row)

chr_ = con.execute('SELECT card_ref,name,home_city,typical_amount FROM
cardholders WHERE card_ref=?', (txn.card_ref,)).fetchone()

holder = Cardholder(*chr_)

rules = [AmountDeviationRule(), UnusualHourRule(), HighRiskCategoryRule(),
VelocityRule()]

print(txn)

for r in rules:

    res = r.score(txn, holder, [])

    print(f'  {res.rule_name:<24} {res.score:>5}  {res.reason}')

"
```

7. Note what inheritance bought: adding a fifth rule requires no change to any caller.

Test it

FraudRule cannot be instantiated; all four rules implement score() and return a RuleResult; a known-fraud transaction produces non-zero scores from the relevant rules.

Note: Full commands and screenshots are in labs/lab-07-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 8 — Compose Rules into an Engine

Learning outcome: Use composition to assemble behaviour from independent parts.

Goal: The learner builds a RuleEngine that HAS-A list of rules rather than inheriting from any of them, producing a weighted composite score with a full explanation of every contributing rule.

What you'll build

A RuleEngine returning a ScreeningResult with a composite score and per-rule reasons (Tools: uv, Python 3.12.)

Step-by-step

1. Write the engine — note it inherits from nothing; it composes

```
# engine.py
from dataclasses import dataclass, field
from models import Transaction
from cardholder import Cardholder
from rules import FraudRule, RuleResult


@dataclass
class ScreeningResult:
    """The engine's decision, with every reason that produced it."""
    txn_id: int
    card_ref: str
    composite_score: float
    flagged: bool
    results: list[RuleResult] = field(default_factory=list)

    def explain(self) -> str:
        head = (f"Transaction {self.txn_id} ({self.card_ref}): "
                f"score {self.composite_score} "
                f"{'FLAGGED' if self.flagged else 'clean'}")
        lines = [f"    - {r.rule_name} {r.score} :: {r.reason}"
                 for r in self.results if r.score > 0]
        return "\n".join([head, *lines])
```

2. Add the engine itself

```
class RuleEngine:
    """Composes independent rules into one weighted decision."""

    def __init__(self, rules: list[FraudRule], threshold: float = 0.55):
        if not rules:
```

```
            raise ValueError("at least one rule is required")
        self.rules = rules
        self.threshold = threshold

    def screen(self, txn: Transaction, holder: Cardholder,
               history: list[Transaction]) -> ScreeningResult:
        results = [r.score(txn, holder, history) for r in self.rules]
        total_weight = sum(r.weight for r in self.rules)
        composite = sum(res.score * rule.weight
                        for res, rule in zip(results, self.rules)) /
total_weight
        composite = round(composite, 3)
        return ScreeningResult(txn.id, txn.card_ref, composite,
                               composite >= self.threshold, results)
```

3. Screen a known-fraud transaction and read the explanation

```
uv run python -c "
import sqlite3
from models import Transaction
from cardholder import Cardholder
from rules import AmountDeviationRule, UnusualHourRule,
HighRiskCategoryRule, VelocityRule
from engine import RuleEngine
con = sqlite3.connect('cardguard.db')
row = con.execute('SELECT * FROM transactions WHERE is_known_fraud=1 ORDER
BY amount DESC LIMIT 1').fetchone()
txn = Transaction.from_row(row)
holder = Cardholder(*con.execute('SELECT
card_ref,name,home_city,typical_amount FROM cardholders WHERE card_ref=?',
(txn.card_ref,)).fetchone())
eng = RuleEngine([AmountDeviationRule(), UnusualHourRule(),
HighRiskCategoryRule(), VelocityRule()])
```

```
print(eng.screen(txn, holder, []).explain())

"
```

4. Screen a normal transaction and confirm it stays clean

```
uv run python -c "

import sqlite3

from models import Transaction

from cardholder import Cardholder

from rules import AmountDeviationRule, UnusualHourRule,
HighRiskCategoryRule, VelocityRule

from engine import RuleEngine

con = sqlite3.connect('cardguard.db')

row = con.execute("SELECT * FROM transactions WHERE is_known_fraud=0 AND
merchant_category='grocery' LIMIT 1").fetchone()

txn = Transaction.from_row(row)

holder = Cardholder(*con.execute('SELECT
card_ref,name,home_city,typical_amount FROM cardholders WHERE card_ref=?',
(txn.card_ref,)).fetchone())

eng = RuleEngine([AmountDeviationRule(), UnusualHourRule(),
HighRiskCategoryRule(), VelocityRule()])

print(eng.screen(txn, holder, []).explain())

"
```

5. Reconfigure the engine WITHOUT touching any rule class — composition in action

```
uv run python -c "

from rules import AmountDeviationRule, HighRiskCategoryRule

from engine import RuleEngine

strict = RuleEngine([AmountDeviationRule(alert_factor=4.0),
HighRiskCategoryRule()], threshold=0.35)

print('rules:', [r.name for r in strict.rules], 'threshold:',
strict.threshold)

"
```

6. Ask your AI assistant to add a fifth rule and confirm the engine needs no edit

```
Ask the assistant: Add a RoundAmountRule to rules.py that subclasses
FraudRule with weight 0.4 and returns score 0.5 when the transaction amount
is an exact multiple of 100, since round-number testing charges are a
common card-testing signal. Do not modify RuleEngine.
```

7. Note the design principle: inheritance shares WHAT rules are; composition decides WHICH rules run.

Test it

The engine flags the known-fraud transaction with a readable explanation, leaves a grocery transaction clean, and can be reconfigured with different rules and thresholds without editing any rule class.

Note: Full commands and screenshots are in labs/lab-08-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Topic 03 — Data Analytics with pandas

DataFrames · Cleaning · Transformation · GroupBy · Aggregation · Reporting

Key concepts

- A DataFrame is a labelled 2-D table; a Series is one column. Most analytics is selecting, filtering and reshaping these.
- Real data is dirty: missing values, wrong dtypes and duplicates must be handled explicitly before analysis.
- split-apply-combine (groupby then aggregate) answers most business questions about a dataset.
- An analytics pipeline should be a set of small, testable functions — not one long script.

Lab 9 — Load and Explore Transactions with pandas

Learning outcome: Read SQL into a DataFrame and profile it.

Goal: The learner loads the transaction table into pandas, inspects dtypes and null counts, and produces a first profile of spending — the step every analytics task starts with.

What you'll build

A loaded, correctly-typed transactions DataFrame plus a spend profile by category (Tools: uv, pandas 3.0, sqlite3.)

Step-by-step

1. Add pandas and load the table

```
uv add pandas

uv run python -c "

import sqlite3, pandas as pd

con = sqlite3.connect('cardguard.db')

df = pd.read_sql_query('SELECT * FROM transactions', con)

print(df.shape)

print(df.head())

"
```

2. Inspect dtypes — note ts arrived as text, not datetime

```
uv run python -c "

import sqlite3, pandas as pd

con = sqlite3.connect('cardguard.db')

df = pd.read_sql_query('SELECT * FROM transactions', con)

print(df.dtypes)

print(df.isna().sum())

"
```

3. Write a loader that fixes the types once, so no downstream code has to

```
# analytics.py

import sqlite3

import pandas as pd


def load_transactions(db_path: str = "cardguard.db") -> pd.DataFrame:

    """Load transactions with correct dtypes."""

    con = sqlite3.connect(db_path)

    try:

        df = pd.read_sql_query("SELECT * FROM transactions", con)

    finally:

        con.close()          # always closes, even if the query raises
```

```
    df["ts"] = pd.to_datetime(df["ts"])
    df["merchant_category"] = df["merchant_category"].astype("category")
    df["hour"] = df["ts"].dt.hour
    df["date"] = df["ts"].dt.date
    return df
```

4. Profile spend by merchant category — split-apply-combine

```
uv run python -c "
from analytics import load_transactions
df = load_transactions()
profile = (df.groupby('merchant_category', observed=True)['amount']
             .agg(['count', 'sum', 'mean', 'max']).round(2)
             .sort_values('sum', ascending=False))
print(profile)
"
```

5. Answer a real business question: which hours carry the most spend?

```
uv run python -c "
from analytics import load_transactions
df = load_transactions()
by_hour = df.groupby('hour')['amount'].agg(['count','sum']).round(2)
print(by_hour.tail(8))
print('\novernight share:',
round(100*df[df.hour<6]['amount'].sum()/df['amount'].sum(),2), '%')
"
```

6. See the pandas 3.0 Copy-on-Write behaviour that AI assistants get wrong

```
uv run python -c "
import pandas as pd
df = pd.DataFrame({'a':[1,2,3],'b':[10,20,30]})
# The pandas 1.x idiom an assistant will often generate:
df[df.a > 1]['b'] = 0
print(df)   # unchanged under Copy-on-Write — the write went to a temporary
```

```
# The correct form:

df.loc[df.a > 1, 'b'] = 0

print(df)

"
```

7. Note the lesson: verify generated pandas against your installed version, not against a tutorial.

Test it

load_transactions returns 10,851 rows with ts as datetime64; the category profile and hourly breakdown print; the learner can state why chained assignment silently fails.

Note: Full commands and screenshots are in labs/lab-09-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 10 — Handle Errors with Exceptions

Learning outcome: Use try/except/else/finally and raise meaningful custom exceptions.

Goal: The learner makes the loader robust: catching the specific failures that actually occur, defining a custom exception hierarchy for the domain, and guaranteeing cleanup with finally. A screening service that crashes on one bad row fails open — this lab prevents that.

What you'll build

A domain exception hierarchy plus a validated, fail-safe loader (Tools: uv, pandas, sqlite3, Python exceptions.)

Step-by-step

1. Define exceptions that describe the DOMAIN, not the plumbing

```
# errors.py

class CardGuardError(Exception):

    """Base class for every CardGuard failure — callers can catch this one
type."""


class DataSourceError(CardGuardError):

    """The transaction store could not be read."""
```

```
class ValidationError(CardGuardError):
    """A transaction failed validation before scoring."""
    def __init__(self, field: str, value, message: str):
        self.field, self.value = field, value
        super().__init__(f"{field}={value!r}: {message}")


class RuleExecutionError(CardGuardError):
    """A scoring rule raised while screening a transaction."""
```

2. Catch the SPECIFIC errors that happen, and translate them to domain errors

```
# analytics.py (updated)
import sqlite3
from pathlib import Path
import pandas as pd
from errors import DataSourceError


def load_transactions(db_path: str = "cardguard.db") -> pd.DataFrame:
    """Load transactions, raising DataSourceError on any read failure."""
    if not Path(db_path).exists():
        raise DataSourceError(f"database not found: {db_path}")
    con = None
    try:
        con = sqlite3.connect(db_path)
        df = pd.read_sql_query("SELECT * FROM transactions", con)
    except sqlite3.DatabaseError as exc:
        raise DataSourceError(f"could not read {db_path}: {exc}") from exc
    else:
        df["ts"] = pd.to_datetime(df["ts"])
        df["hour"] = df["ts"].dt.hour
        return df
```

```
    finally:
        if con is not None:
            con.close()      # runs on success AND on failure
```

3. Prove each failure path raises the right domain error

```
uv run python -c "
from analytics import load_transactions
from errors import DataSourceError
try:
    load_transactions('no_such.db')
except DataSourceError as e:
    print('missing file ->', e)
open('corrupt.db','wb').write(b'not a database at all')
try:
    load_transactions('corrupt.db')
except DataSourceError as e:
    print('corrupt file ->', type(e).__name__)
"
```

4. Validate a transaction and raise ValidationError carrying the offending field

```
# validate.py
from errors import ValidationError


VALID_CATEGORIES =
{"grocery","fuel","restaurant","electronics","online_gaming",

"jewellery","crypto_exchange","gift_cards","pharmacy","transport"}


def validate_txn(record: dict) -> dict:
    """Raise ValidationError on the first problem found."""
    amount = record.get("amount")
    if amount is None:
```

```
        raise ValidationError("amount", amount, "is required")
    if not isinstance(amount, (int, float)):
        raise ValidationError("amount", amount, "must be numeric")
    if amount <= 0:
        raise ValidationError("amount", amount, "must be positive")
    if amount > 1_000_000:
        raise ValidationError("amount", amount, "exceeds the maximum single
charge")
    cat = record.get("merchant_category")
    if cat not in VALID_CATEGORIES:
        raise ValidationError("merchant_category", cat, "is not a known
category")
    return record
```

5. Run a batch where some records are bad — the batch must NOT die on the first failure

```
uv run python -c "
from validate import validate_txn
from errors import ValidationError
batch = [
  {'amount': 120.0, 'merchant_category': 'grocery'},
  {'amount': -5.0,  'merchant_category': 'fuel'},
  {'amount': 'abc', 'merchant_category': 'grocery'},
  {'amount': 90.0,  'merchant_category': 'casino'},
  {'amount': 45.0,  'merchant_category': 'transport'},
]
ok, rejected = [], []
for i, rec in enumerate(batch):
    try:
        ok.append(validate_txn(rec))
    except ValidationError as e:
```

```
        rejected.append((i, str(e)))

print(f'accepted {len(ok)}, rejected {len(rejected)}')

for i, msg in rejected:

    print(f'  row {i}: {msg}')

"
```

6. Contrast with the anti-pattern — a bare except hides the bug you needed to see

```
uv run python -c "

# ANTI-PATTERN: never do this

try:

    result = 1 / 0

except:              # catches everything, including typos and
KeyboardInterrupt

    result = 0

print('silently wrong:', result)


# Correct: name the exception you expect and can handle

try:

    result = 1 / 0

except ZeroDivisionError as e:

    print('handled explicitly:', e)

"
```

7. Ask your assistant to add a rule-level guard so one broken rule cannot stop a screen

```
Ask the assistant: In RuleEngine.screen, wrap each rule call in try/except
so that if one rule raises, the engine records a RuleExecutionError message
on the result and continues scoring with the remaining rules. Never use a
bare except.
```

Test it

Missing and corrupt databases raise DataSourceError; the 5-record batch accepts 2 and rejects 3 with field-level messages; the learner can explain why bare except is unsafe.

Note: Full commands and screenshots are in labs/lab-10-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 11 — Build Per-Cardholder Baselines with GroupBy

Learning outcome: Compute per-group statistics with split-apply-combine.

Goal: The learner computes each cardholder's own spending baseline and deviation for every transaction — the statistical foundation the AmountDeviationRule needs, done for 40 cardholders at once instead of one at a time.

What you'll build

A per-cardholder baseline table and a deviation column on every transaction (Tools: uv, pandas 3.0.)

Step-by-step

1. Compute one baseline per cardholder

```
uv run python -c "

from analytics import load_transactions

df = load_transactions()

base = df.groupby('card_ref')
['amount'].agg(['count','mean','median','std']).round(2)

print(base.head())

print('cardholders:', len(base))

"
```

2. Join each transaction to its cardholder baseline with transform — same shape as the original

```
# analytics.py (add)

def add_deviation(df):

    """Attach each transaction's deviation from its own cardholder
baseline."""

    df = df.copy()

    df["baseline"] = df.groupby("card_ref")["amount"].transform("median")

    df["dev_factor"] = (df["amount"] / df["baseline"]).round(2)

    return df
```

3. Find the biggest deviations and check them against the seeded fraud flag

```
uv run python -c "

from analytics import load_transactions, add_deviation

df = add_deviation(load_transactions())

top = df.nlargest(10, 'dev_factor')
[['id','card_ref','amount','baseline','dev_factor','merchant_category','is_
known_fraud']]

print(top.to_string(index=False))

print('\nof the top 10 deviations,', int(top.is_known_fraud.sum()), 'are
seeded fraud')

"
```

4. Measure how well deviation alone separates fraud from normal

```
uv run python -c "

from analytics import load_transactions, add_deviation

df = add_deviation(load_transactions())

print(df.groupby('is_known_fraud')['dev_factor'].describe().round(2)
[['count','mean','50%','max']])

"
```

5. Use median not mean for the baseline — prove why with an outlier

```
uv run python -c "

import pandas as pd

s = pd.Series([100,110,95,105,20000])

print('mean  ', round(s.mean(),2), '<- dragged up by the fraud itself')

print('median', round(s.median(),2), '<- robust to the outlier')

"
```

6. Note the analytics lesson: the statistic you choose changes what you can detect.

Test it

Baselines compute for all 40 cardholders; dev_factor is attached to every row; the top deviations are dominated by seeded fraud; the learner can justify median over mean.

Note: Full commands and screenshots are in labs/lab-11-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 12 — Detect Velocity Bursts with Rolling Time Windows

Learning outcome: Analyse ordered time-series data per group.

Goal: The learner sorts transactions per cardholder and counts how many occur inside a rolling 10-minute window — the history the VelocityRule needs, which cannot be computed one transaction at a time.

What you'll build

A txn_count_10min column plus the burst transactions it exposes (Tools: uv, pandas 3.0, rolling windows.)

Step-by-step

1. Sort by cardholder and time — order is a precondition for any window function

```
# analytics.py (add)

def add_velocity(df, window: str = "10min"):

    """Count transactions per cardholder inside a rolling time window."""

    df = df.sort_values(["card_ref", "ts"]).copy()

    counts = (df.set_index("ts")

                .groupby("card_ref")["amount"]

                .rolling(window).count()

                .reset_index(name="txn_count_10min"))

    df = df.merge(counts, on=["card_ref", "ts"], how="left")

    return df
```

2. Find the bursts

```
uv run python -c "

from analytics import load_transactions, add_velocity

df = add_velocity(load_transactions())

bursts = df[df.txn_count_10min >= 4]

print('burst transactions:', len(bursts))

print(bursts[['card_ref','ts','amount','merchant_category','txn_count_10min
','is_known_fraud']].head(12).to_string(index=False))

"
```

3. Check the hit rate of velocity as a signal on its own

```
uv run python -c "

from analytics import load_transactions, add_velocity

df = add_velocity(load_transactions())

b = df[df.txn_count_10min >= 4]

print(f'{int(b.is_known_fraud.sum())} of {len(b)} burst transactions are
seeded fraud')

print('precision:', round(100*b.is_known_fraud.mean(),1), '%')

"
```

4. Inspect one full burst to see the pattern a fraudster leaves

```
uv run python -c "

from analytics import load_transactions, add_velocity

df = add_velocity(load_transactions())

card = df[df.txn_count_10min >= 5].card_ref.iloc[0]

win = df[df.card_ref == card].nlargest(8, 'txn_count_10min')
[['ts','amount','merchant_category','txn_count_10min']]

print(f'card {card}'); print(win.sort_values('ts').to_string(index=False))

"
```

5. Compute the geographic-impossibility signal the engine is still missing

```
# analytics.py (add)

import numpy as np


def add_geo_velocity(df):

    """Implied travel speed (km/h) between consecutive transactions."""

    df = df.sort_values(["card_ref", "ts"]).copy()

    g = df.groupby("card_ref")

    lat1, lon1 = np.radians(g["lat"].shift()), np.radians(g["lon"].shift())

    lat2, lon2 = np.radians(df["lat"]), np.radians(df["lon"])

    dlat, dlon = lat2 - lat1, lon2 - lon1

    a = np.sin(dlat/2)**2 + np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2
```

```
    km = 6371 * 2 * np.arcsin(np.sqrt(a))

    hours = g["ts"].diff().dt.total_seconds() / 3600

    df["implied_kmh"] = (km / hours.replace(0, np.nan)).round(1)

    return df
```

6. Find the physically impossible journeys

```
uv run python -c "

from analytics import load_transactions, add_geo_velocity

df = add_geo_velocity(load_transactions())

imp = df[df.implied_kmh > 1000]

print('impossible journeys:', len(imp))

print(imp[['card_ref','ts','city','amount','implied_kmh','is_known_fraud']]
.to_string(index=False))

"
```

7. Note: velocity and geography are only visible ACROSS rows. Row-at-a-time scoring cannot see them.

Test it

Velocity bursts and impossible journeys are both detected, dominated by seeded fraud, and the learner can explain why these signals require sorted per-group windows.

Note: Full commands and screenshots are in labs/lab-12-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Topic 04 — Data Modelling with Pydantic and FastAPI

Typed Models · Validation · Endpoints · Request/Response Schemas · Auto Docs

Key concepts

- Pydantic models declare the shape of data with Python type hints and validate it at runtime, failing fast on bad input.
- FastAPI turns typed Python functions into HTTP endpoints, deriving validation and OpenAPI docs from the annotations.
- Separate request models from response models so the API contract is explicit and safe to change.
- The analytics layer stays plain Python; FastAPI is a thin transport layer over it.

Lab 13 — Replace Hand-Written Validation with Pydantic Models

Learning outcome: Declare data shape with type hints and validate at runtime.

Goal: The learner replaces the manual validate_txn() from Topic 3 with a Pydantic model, getting type coercion, field constraints and structured error reporting for free — far less code, far better errors.

What you'll build

TransactionIn and ScreeningOut Pydantic models with field constraints (Tools: uv, Pydantic v2.)

Step-by-step

1. Add Pydantic to the project

```
uv add pydantic
```

2. Declare the model — the constraints ARE the validation

```
# schemas.py

from datetime import datetime

from typing import Literal

from pydantic import BaseModel, Field, field_validator


CATEGORIES =
Literal["grocery","fuel","restaurant","electronics","online_gaming",


"jewellery","crypto_exchange","gift_cards","pharmacy","transport"]


class TransactionIn(BaseModel):

    """One transaction submitted for screening."""

    card_ref: str = Field(pattern=r"^CH\d{4}$", description="Cardholder
reference")

    ts: datetime

    amount: float = Field(gt=0, le=1_000_000, description="Charge amount in
SGD")

    merchant_category: CATEGORIES

    city: str = Field(min_length=2, max_length=64)
```

```
    lat: float = Field(ge=-90, le=90)
    lon: float = Field(ge=-180, le=180)


    @field_validator("city")
    @classmethod
    def title_case_city(cls, v: str) -> str:
        return v.strip().title()
```

3. Compare against Topic 3 — one model replaces the whole hand-written validator

```
uv run python -c "
from schemas import TransactionIn
good = TransactionIn(card_ref='CH0001', ts='2026-04-15T02:14:00',
amount=4820.50,
                     merchant_category='jewellery', city='singapore',
lat=1.3521, lon=103.8198)
print(good)
print('city normalised to:', good.city)
print('ts coerced to:', type(good.ts).__name__)
"
```

4. Read a validation failure — Pydantic reports EVERY problem, not just the first

```
uv run python -c "
from pydantic import ValidationError
from schemas import TransactionIn
try:
    TransactionIn(card_ref='BAD', ts='not-a-date', amount=-5,
                  merchant_category='casino', city='X', lat=999, lon=0)
except ValidationError as e:
    for err in e.errors():
        print(f\"  {'.'.join(str(x) for x in err['loc']):<20} {err['msg']}
\")
"
```

5. Define the response model separately — request and response are different contracts

```
# schemas.py (add)

class RuleHit(BaseModel):

    rule_name: str

    score: float = Field(ge=0, le=1)

    reason: str


class ScreeningOut(BaseModel):

    """What the screening service returns."""

    card_ref: str

    composite_score: float = Field(ge=0, le=1)

    flagged: bool

    decision: Literal["approve", "review", "decline"]

    hits: list[RuleHit] = []

    errors: list[str] = []
```

6. Serialise to JSON — the wire format comes free

```
uv run python -c "

from schemas import ScreeningOut, RuleHit

out = ScreeningOut(card_ref='CH0023', composite_score=0.933, flagged=True,
decision='decline',

                   hits=[RuleHit(rule_name='AmountDeviationRule',
score=1.0, reason='26.89x baseline'),

                         RuleHit(rule_name='UnusualHourRule', score=0.8,
reason='02:00')])

print(out.model_dump_json(indent=2))

"
```

7. Load real database rows THROUGH the model to catch any dirty data

```
uv run python -c "

import sqlite3

from pydantic import ValidationError
```

```
from schemas import TransactionIn

con = sqlite3.connect('cardguard.db')

con.row_factory = sqlite3.Row

ok = bad = 0

for r in con.execute('SELECT * FROM transactions LIMIT 500'):

    try:

        TransactionIn(**{k: r[k] for k in
('card_ref','ts','amount','merchant_category','city','lat','lon')})

        ok += 1

    except ValidationError:

        bad += 1

print(f'validated {ok}, rejected {bad}')

"
```

8. Note the trade: hand-written validation is explicit but verbose; Pydantic is declarative and reports all errors at once.

Test it

Valid input constructs and normalises; invalid input reports every field error; ScreeningOut serialises to JSON; 500 real rows validate cleanly.

Note: Full commands and screenshots are in labs/lab-13-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 14 — Expose the Screening Engine as a FastAPI Endpoint

Learning outcome: Turn typed Python functions into HTTP endpoints.

Goal: The learner wraps the Topic 2 RuleEngine in a FastAPI POST endpoint. The engine is not modified at all — FastAPI is a thin transport layer over logic that already works.

What you'll build

A running API with POST /screen, GET /health and automatic OpenAPI docs (Tools: uv, FastAPI, uvicorn, Pydantic.)

Step-by-step

1. Add the web dependencies

```
uv add fastapi uvicorn[standard] httpx
```

2. Write the application — note how little code the endpoint needs

```
# main.py

import sqlite3

from fastapi import FastAPI, HTTPException

from schemas import TransactionIn, ScreeningOut, RuleHit

from models import Transaction

from cardholder import Cardholder

from rules import AmountDeviationRule, UnusualHourRule,
HighRiskCategoryRule, VelocityRule

from engine import RuleEngine


app = FastAPI(title="CardGuard Screening API", version="1.0.0")


ENGINE = RuleEngine([AmountDeviationRule(), UnusualHourRule(),

                     HighRiskCategoryRule(), VelocityRule()])


def get_holder(card_ref: str) -> Cardholder:

    con = sqlite3.connect("cardguard.db")

    try:

        row = con.execute("SELECT card_ref,name,home_city,typical_amount "

                          "FROM cardholders WHERE card_ref=?",
(card_ref,)).fetchone()

    finally:

        con.close()

    if row is None:

        raise HTTPException(status_code=404, detail=f"unknown card_ref
{card_ref}")

    return Cardholder(*row)
```

```
def decide(score: float) -> str:
    if score >= 0.80: return "decline"
    if score >= 0.55: return "review"
    return "approve"


@app.get("/health")
def health() -> dict:
    return {"status": "ok"}


@app.post("/screen", response_model=ScreeningOut)
def screen(txn_in: TransactionIn) -> ScreeningOut:
    """Score one transaction and return the decision with reasons."""
    holder = get_holder(txn_in.card_ref)
    txn = Transaction(id=0, card_ref=txn_in.card_ref, ts=txn_in.ts,
                      amount=txn_in.amount,
merchant_category=txn_in.merchant_category,
                      city=txn_in.city, lat=txn_in.lat, lon=txn_in.lon)
    result = ENGINE.screen(txn, holder, [])
    return ScreeningOut(card_ref=result.card_ref,
                        composite_score=result.composite_score,
                        flagged=result.flagged,
                        decision=decide(result.composite_score),
                        hits=[RuleHit(rule_name=r.rule_name, score=r.score,
reason=r.reason)
                              for r in result.results if r.score > 0],
                        errors=result.errors)
```

3. Start the server

```
uv run uvicorn main:app --reload --port 8000
```

4. Open the auto-generated interactive docs — you wrote no OpenAPI spec

```
open http://127.0.0.1:8000/docs
```

5. Screen a clearly fraudulent transaction

```
curl -s -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" -d '{"card_ref":"CH0023","ts":"2026-04-
15T02:14:00","amount":5128.33,"merchant_category":"jewellery","city":"Singa
pore","lat":1.3521,"lon":103.8198}' | python3 -m json.tool
```

6. Screen a normal grocery run and compare the decision

```
curl -s -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" -d '{"card_ref":"CH0001","ts":"2026-04-
15T10:30:00","amount":86.40,"merchant_category":"grocery","city":"Singapore
","lat":1.3521,"lon":103.8198}' | python3 -m json.tool
```

7. Send invalid input — FastAPI rejects it with 422 before your code runs

```
curl -s -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" -d '{"card_ref":"NOPE","ts":"2026-04-
15T10:30:00","amount":-5,"merchant_category":"casino","city":"S","lat":1.35
,"lon":103.8}' | python3 -m json.tool
```

8. Send an unknown card and confirm the 404 path

```
curl -s -i -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" -d '{"card_ref":"CH9999","ts":"2026-04-
15T10:30:00","amount":50,"merchant_category":"grocery","city":"Singapore","
lat":1.35,"lon":103.8}' | head -1
```

Test it

GET /health returns ok; the fraud transaction returns decline with reasons; the grocery transaction returns approve; bad input returns 422; an unknown card returns 404.

Note: Full commands and screenshots are in labs/lab-14-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 15 — Persist Screening Decisions to SQLite

Learning outcome: Write application state to a database from the API layer.

Goal: The learner adds a repository layer that records every screening decision, so the service builds an audit trail — a regulatory requirement for real fraud systems, and the data the Topic 5 dashboard reads.

What you'll build

A screenings table plus a repository module with save and query functions (Tools: uv, sqlite3, FastAPI.)

Step-by-step

1. Create the audit table

```
# repository.py

import sqlite3

import json

from contextlib import contextmanager

from pathlib import Path


DB = "cardguard.db"


@contextmanager

def connect(db_path: str = DB):

    """Connection that always commits or rolls back, and always closes."""

    con = sqlite3.connect(db_path)

    con.row_factory = sqlite3.Row

    try:

        yield con

        con.commit()

    except Exception:

        con.rollback()

        raise

    finally:

        con.close()


def init_schema(db_path: str = DB) -> None:

    with connect(db_path) as con:

        con.execute("""

            CREATE TABLE IF NOT EXISTS screenings (
```

```
                id INTEGER PRIMARY KEY AUTOINCREMENT,

                card_ref TEXT NOT NULL,

                ts TEXT NOT NULL,

                amount REAL NOT NULL,

                merchant_category TEXT NOT NULL,

                composite_score REAL NOT NULL,

                decision TEXT NOT NULL,

                hits_json TEXT NOT NULL,

                screened_at TEXT NOT NULL DEFAULT (datetime('now')))""")

        con.execute("CREATE INDEX IF NOT EXISTS idx_scr_card ON
screenings(card_ref)")
```

2. Add save and query functions — parameterised, never string-formatted

```
# repository.py (add)

def save_screening(txn_in, result, db_path: str = DB) -> int:

    """Record one decision; returns the new row id."""

    with connect(db_path) as con:

        cur = con.execute(

            "INSERT INTO screenings (card_ref, ts, amount,
merchant_category,"

            " composite_score, decision, hits_json) VALUES
(?,?,?,?,?,?,?)",

            (txn_in.card_ref, txn_in.ts.isoformat(), txn_in.amount,

             txn_in.merchant_category, result.composite_score,
result.decision,

             json.dumps([h.model_dump() for h in result.hits])))

        return cur.lastrowid


def recent_screenings(limit: int = 20, db_path: str = DB) -> list[dict]:

    with connect(db_path) as con:

        rows = con.execute(
```

```
            "SELECT * FROM screenings ORDER BY id DESC LIMIT ?",
(limit,)).fetchall()

    return [dict(r) for r in rows]


def decision_counts(db_path: str = DB) -> dict[str, int]:

    with connect(db_path) as con:

        rows = con.execute(

            "SELECT decision, COUNT(*) n FROM screenings GROUP BY
decision").fetchall()

    return {r["decision"]: r["n"] for r in rows}
```

3. Show why parameterised queries matter — the injection an f-string would allow

```
uv run python -c "

card = \"CH0001'; DROP TABLE screenings; --\"

print('UNSAFE would build:')

print(f\"  SELECT * FROM screenings WHERE card_ref = '{card}'\")

print('SAFE passes the value separately: con.execute(sql, (card,))')

"
```

4. Wire persistence into the endpoint

```
# main.py (add)

from repository import init_schema, save_screening, recent_screenings,
decision_counts


@app.on_event("startup")

def _startup() -> None:

    init_schema()


# inside screen(), just before returning:

#     save_screening(txn_in, out)


@app.get("/screenings")
```

```
def list_screenings(limit: int = 20) -> list[dict]:

    """Recent decisions, newest first — the audit trail."""

    return recent_screenings(limit)


@app.get("/stats")

def stats() -> dict:

    """Decision counts for the dashboard."""

    return decision_counts()
```

5. Restart and post several transactions to build history

```
for amt in 86.40 5128.33 240.00 3900.00; do

  curl -s -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" \

    -d "{\"card_ref\":\"CH0001\",\"ts\":\"2026-04-15T02:30:00\",\"amount\":
$amt,\"merchant_category\":\"electronics\",\"city\":\"Singapore\",
\"lat\":1.3521,\"lon\":103.8198}" > /dev/null

done

curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

6. Read the audit trail back

```
curl -s 'http://127.0.0.1:8000/screenings?limit=5' | python3 -m json.tool
```

7. Verify rollback works — a failed write must leave no partial row

```
uv run python -c "

import repository as repo

repo.init_schema()

before = len(repo.recent_screenings(1000))

try:

    with repo.connect() as con:

        con.execute('INSERT INTO screenings
(card_ref,ts,amount,merchant_category,composite_score,decision,hits_json)
VALUES (?,?,?,?,?,?,?)',

                    ('CH0001','2026-04-
15T10:00:00',50.0,'grocery',0.1,'approve','[]'))

        raise RuntimeError('simulated failure after insert')
```

```
except RuntimeError as e:

    print('caught:', e)

after = len(repo.recent_screenings(1000))

print(f'rows before {before}, after {after} -> rolled back: {before ==
after}')

"
```

Test it

Screenings persist and are queryable via /screenings and /stats; the injection demo shows why parameters are used; the rollback test proves the row count is unchanged after a mid-transaction failure.

Note: Full commands and screenshots are in labs/lab-15-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 16 — Test the API with pytest and httpx

Learning outcome: Write automated tests for a FastAPI application.

Goal: The learner writes a test suite covering the happy path, validation failures and persistence — so that later refactoring and deployment changes are provably safe.

What you'll build

A passing pytest suite against the screening API (Tools: uv, pytest, FastAPI TestClient.)

Step-by-step

1. Add the test dependencies

```
uv add --dev pytest httpx
```

2. Write tests for the decision paths

```
# test_api.py

import pytest

from fastapi.testclient import TestClient

from main import app


client = TestClient(app)
```

```
def post(**overrides):
    body = {"card_ref": "CH0001", "ts": "2026-04-15T10:30:00", "amount":
86.40,
            "merchant_category": "grocery", "city": "Singapore",
            "lat": 1.3521, "lon": 103.8198}
    body.update(overrides)
    return client.post("/screen", json=body)


def test_health():
    assert client.get("/health").json() == {"status": "ok"}


def test_normal_transaction_is_approved():
    r = post()
    assert r.status_code == 200
    assert r.json()["decision"] == "approve"


def test_large_offhours_transaction_is_escalated():
    r = post(amount=5128.33, merchant_category="jewellery", ts="2026-04-
15T02:14:00")
    body = r.json()
    assert body["decision"] in {"review", "decline"}
    assert body["hits"], "an escalated decision must carry its reasons"
```

3. Add tests for the failure paths

```
# test_api.py (add)
@pytest.mark.parametrize("bad", [
    {"amount": -5},
    {"amount": 0},
    {"card_ref": "NOPE"},
```

```
    {"merchant_category": "casino"},
    {"lat": 999},
])
def test_invalid_input_is_rejected(bad):
    assert post(**bad).status_code == 422


def test_unknown_card_returns_404():
    assert post(card_ref="CH9999").status_code == 404


def test_screening_is_persisted():
    before = len(client.get("/screenings?limit=1000").json())
    post()
    after = len(client.get("/screenings?limit=1000").json())
    assert after == before + 1
```

4. Run the suite

```
uv run pytest -v
```

5. Check what the tests actually cover

```
uv add --dev pytest-cov
uv run pytest --cov=. --cov-report=term-missing
```

6. Ask your assistant to write a test for a gap — then verify it FAILS first

```
Ask the assistant: Write a pytest test asserting that POST /screen with
amount exactly 1000000 succeeds but 1000001 returns 422, matching the
Field(le=1_000_000) constraint in TransactionIn.
```

7. Note the discipline: a test you have never seen fail is a test you cannot trust.

Test it

All tests pass; invalid inputs are parameterised and rejected with 422; the persistence test proves a row is written per screening.

Note: Full commands and screenshots are in labs/lab-16-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Topic 05 — Packaging and Deployment

uv Lockfiles · Configuration · Containers · Deploying a Service

Key concepts

- uv.lock pins exact versions so the application installs identically in development and production.
- Configuration belongs in environment variables, never hard-coded — secrets must never be committed.
- A container image bundles the interpreter, dependencies and application code into one deployable artifact.
- A deployment is not done until the running service has been verified with a real request against a health endpoint.

Lab 17 — Build a Fraud Analyst Dashboard with Streamlit

Learning outcome: Create a data application UI that consumes the API.

Goal: The learner builds the third tier: a Streamlit dashboard a fraud analyst actually uses — review queue, screening form and trend charts — calling the FastAPI service rather than reaching into the database.

What you'll build

A running Streamlit dashboard with live screening, a review queue and charts (Tools: uv, Streamlit, FastAPI, httpx, pandas.)

Step-by-step

1. Add Streamlit

```
uv add streamlit httpx
```

2. Build the dashboard shell and the live screening form

```
# app.py

import httpx

import pandas as pd

import streamlit as st


API = st.secrets.get("api_url", "http://127.0.0.1:8000")


st.set_page_config(page_title="CardGuard", page_icon="\U0001F6E1",
layout="wide")
```

```
st.title("CardGuard — Fraud Screening Console")


with st.sidebar:
    st.header("Screen a transaction")
    card_ref = st.text_input("Card reference", "CH0001")
    amount = st.number_input("Amount (SGD)", min_value=0.01, value=86.40,
step=10.0)
    category = st.selectbox("Merchant category",
        ["grocery","fuel","restaurant","electronics","online_gaming",

"jewellery","crypto_exchange","gift_cards","pharmacy","transport"])
    hour = st.slider("Hour of day", 0, 23, 10)
    submitted = st.button("Screen", type="primary")


if submitted:
    payload = {"card_ref": card_ref, "ts": f"2026-04-15T{hour:02d}:30:00",
               "amount": float(amount), "merchant_category": category,
               "city": "Singapore", "lat": 1.3521, "lon": 103.8198}
    try:
        r = httpx.post(f"{API}/screen", json=payload, timeout=10)
        r.raise_for_status()
        out = r.json()
    except httpx.HTTPStatusError as exc:
        st.error(f"API rejected the request ({exc.response.status_code}):
{exc.response.text}")
    except httpx.RequestError:
        st.error("Cannot reach the screening API. Is it running on port
8000?")
    else:
        colour = {"approve": "green", "review": "orange", "decline": "red"}
[out["decision"]]
```

```
        st.markdown(f"### Decision: :{colour}[{out['decision'].upper()}]")
        st.metric("Composite score", out["composite_score"])
        if out["hits"]:
            st.dataframe(pd.DataFrame(out["hits"]), width="stretch")
        else:
            st.info("No rule fired on this transaction.")
```

3. Add the review queue and trend charts

```
# app.py (add)
st.divider()
left, right = st.columns([2, 1])

try:
    rows = httpx.get(f"{API}/screenings", params={"limit": 200},
timeout=10).json()
    stats = httpx.get(f"{API}/stats", timeout=10).json()
except httpx.RequestError:
    st.warning("API unavailable — showing no history.")
    rows, stats = [], {}

with left:
    st.subheader("Recent screenings")
    if rows:
        df = pd.DataFrame(rows)
        df["screened_at"] = pd.to_datetime(df["screened_at"])
        st.dataframe(
df[["card_ref","amount","merchant_category","composite_score","decision","s
creened_at"]],
            width="stretch", height=340)
    else:
```

```
        st.info("No screenings yet — submit one from the sidebar.")


with right:
    st.subheader("Decisions")
    if stats:
        st.bar_chart(pd.Series(stats, name="count"))
        total = sum(stats.values())
        flagged = stats.get("review", 0) + stats.get("decline", 0)
        st.metric("Flag rate", f"{100 * flagged / total:.1f}%" if total
else "—")
```

4. Run both tiers — API in one terminal, UI in another

```
# terminal 1
uv run uvicorn main:app --reload --port 8000


# terminal 2
uv run streamlit run app.py
```

5. Screen a normal transaction in the UI and watch it land in the queue

```
open http://localhost:8501
```

6. Screen a $5,000 jewellery charge at 02:00 and confirm it turns red
7. Stop the API and confirm the UI degrades gracefully instead of crashing

```
# stop uvicorn (Ctrl-C), then click Screen in the UI
```

8. Note the architecture: the UI never touches SQLite. Swapping the database would not change app.py at all.

Test it

The dashboard screens transactions live, colour-codes decisions, lists the review queue and charts decision counts; with the API stopped it shows a clear error rather than a traceback.

Note: Full commands and screenshots are in labs/lab-17-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 18 — Externalise Configuration and Protect Secrets

Learning outcome: Move settings out of code into the environment.

Goal: The learner replaces every hard-coded value with typed settings loaded from the environment, and ensures secrets can never be committed — the change that makes the same image runnable in dev and production.

What you'll build

A typed Settings object, a .env file and a .gitignore that excludes it (Tools: uv, pydantic-settings, python-dotenv.)

Step-by-step

1. Find every hard-coded value in the project

```
grep -rn '127.0.0.1\|8000\|cardguard.db\|0.55\|0.80' --include='*.py' . |
grep -v '.venv'
```

2. Add typed settings

```
uv add pydantic-settings
```

3. Declare configuration as a validated model

```
# config.py

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):

    """All runtime configuration, validated at startup."""

    model_config = SettingsConfigDict(env_file=".env",
env_prefix="CARDGUARD_")


    db_path: str = "cardguard.db"

    api_url: str = "http://127.0.0.1:8000"

    review_threshold: float = 0.55

    decline_threshold: float = 0.80

    velocity_window_minutes: int = 10

    log_level: str = "INFO"


settings = Settings()
```

4. Create .env for local development and EXCLUDE it from git immediately

```
# .env
CARDGUARD_DB_PATH=cardguard.db
CARDGUARD_REVIEW_THRESHOLD=0.55
CARDGUARD_DECLINE_THRESHOLD=0.80
CARDGUARD_LOG_LEVEL=DEBUG
```

5. Add the gitignore entries BEFORE the first commit

```
# .gitignore
.env
.env.*
!.env.example
.venv/
__pycache__/
*.db
.pytest_cache/
```

6. Commit a .env.example instead, so a new joiner knows what to set

```
# .env.example — safe to commit, contains no real values
CARDGUARD_DB_PATH=cardguard.db
CARDGUARD_REVIEW_THRESHOLD=0.55
CARDGUARD_DECLINE_THRESHOLD=0.80
CARDGUARD_LOG_LEVEL=INFO
```

7. Replace the hard-coded thresholds with settings

```
# main.py (replace decide())
from config import settings


def decide(score: float) -> str:
    if score >= settings.decline_threshold: return "decline"
    if score >= settings.review_threshold:  return "review"
    return "approve"
```

8. Prove configuration is live — override without editing any code

```
uv run python -c "

from config import settings

print('default review threshold:', settings.review_threshold)

"

CARDGUARD_REVIEW_THRESHOLD=0.30 uv run python -c "

from config import settings

print('overridden by environment:', settings.review_threshold)

"
```

9. Confirm .env is genuinely ignored

```
git check-ignore -v .env && echo 'SAFE: .env will not be committed'
```

Test it

Settings load from .env; an environment variable overrides them without a code change; git check-ignore confirms .env is excluded; .env.example is committed in its place.

Note: Full commands and screenshots are in labs/lab-18-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 19 — Lock Dependencies and Containerise the API

Learning outcome: Package the application into a reproducible image.

Goal: The learner writes a multi-stage Dockerfile that installs from uv.lock, runs as a non-root user, and produces an image that behaves identically on any machine.

What you'll build

A built Docker image running the screening API with a health check (Tools: uv, Docker, uvicorn.)

Step-by-step

1. Confirm the lockfile is current and committed

```
uv lock --check

git add uv.lock pyproject.toml
```

2. Write the Dockerfile — multi-stage keeps the final image small

```
# Dockerfile
```

```
FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim AS builder

WORKDIR /app

ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy


# Install dependencies first so this layer caches across code changes

COPY pyproject.toml uv.lock ./

RUN uv sync --frozen --no-install-project --no-dev


COPY . .

RUN uv sync --frozen --no-dev


FROM python:3.12-slim-bookworm AS runtime

RUN useradd --create-home --uid 1000 appuser

WORKDIR /app

COPY --from=builder --chown=appuser:appuser /app /app

ENV PATH="/app/.venv/bin:$PATH" PYTHONUNBUFFERED=1

USER appuser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \\

  CMD python -c "import httpx,sys; sys.exit(0 if
httpx.get('http://127.0.0.1:8000/health').status_code==200 else 1)"

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

3. Exclude everything the image does not need

```
# .dockerignore

.venv/

.git/

.env

*.db

__pycache__/
```

```
.pytest_cache/

tests/

*.md
```

4. Build the image

```
docker build -t cardguard-api:1.0 .
```

5. Check the size — multi-stage should keep it well under 300MB

```
docker images cardguard-api:1.0
```

6. Run it, passing configuration as environment variables

```
docker run -d --name cardguard \

  -p 8000:8000 \

  -e CARDGUARD_REVIEW_THRESHOLD=0.55 \

  -e CARDGUARD_LOG_LEVEL=INFO \

  -v "$(pwd)/cardguard.db:/app/cardguard.db" \

  cardguard-api:1.0
```

7. Verify the container is healthy and answering

```
docker ps --filter name=cardguard --format 'table {{.Names}}\t{{.Status}}'

curl -s http://127.0.0.1:8000/health
```

8. Screen a transaction against the CONTAINER, not your laptop's Python

```
curl -s -X POST http://127.0.0.1:8000/screen -H "Content-Type:
application/json" -d '{"card_ref":"CH0023","ts":"2026-04-
15T02:14:00","amount":5128.33,"merchant_category":"jewellery","city":"Singa
pore","lat":1.3521,"lon":103.8198}' | python3 -m json.tool
```

9. Read the logs and confirm it runs as a non-root user

```
docker logs cardguard --tail 20

docker exec cardguard whoami
```

Test it

The image builds, runs as appuser, reports healthy, and returns a decline decision for the fraud transaction posted to the container.

Note: Full commands and screenshots are in labs/lab-19-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Lab 20 — Deploy the Full Stack with Docker Compose

Learning outcome: Run and verify a multi-service application.

Goal: The learner composes the API and the Streamlit UI into one stack with a shared volume and dependency ordering, then verifies the deployment end to end — the final integration of everything built in Topics 1-5.

What you'll build

A running two-service stack verified with a real screening through the UI (Tools: Docker Compose, FastAPI, Streamlit, SQLite.)

Step-by-step

1. Write a Dockerfile for the UI tier

```
# Dockerfile.ui

FROM ghcr.io/astral-sh/uv:python3.12-bookworm-slim

WORKDIR /app

ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy

COPY pyproject.toml uv.lock ./

RUN uv sync --frozen --no-dev

COPY . .

ENV PATH="/app/.venv/bin:$PATH"

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--
server.port=8501"]
```

2. Compose the stack

```
# compose.yaml

services:

  api:

    build: .

    ports: ["8000:8000"]

    environment:

      CARDGUARD_DB_PATH: /data/cardguard.db

      CARDGUARD_REVIEW_THRESHOLD: "0.55"
```

```
      CARDGUARD_DECLINE_THRESHOLD: "0.80"
    volumes: ["cardguard-data:/data"]
    healthcheck:
      test: ["CMD", "python", "-c", "import httpx,sys; sys.exit(0 if
httpx.get('http://127.0.0.1:8000/health').status_code==200 else 1)"]
      interval: 10s
      timeout: 3s
      retries: 5

  ui:
    build:
      context: .
      dockerfile: Dockerfile.ui
    ports: ["8501:8501"]
    environment:
      CARDGUARD_API_URL: http://api:8000
    depends_on:
      api:
        condition: service_healthy

volumes:
  cardguard-data:
```

3. Point the UI at the service name — inside the network it is not localhost

```
# app.py (replace the API constant)
import os
API = os.getenv("CARDGUARD_API_URL", "http://127.0.0.1:8000")
```

4. Bring the stack up

```
docker compose up --build -d
```

5. Confirm the UI waited for the API to become healthy

```
docker compose ps

docker compose logs api --tail 5
```

6. Seed the database inside the running container

```
docker compose exec api python mockdata.py

docker compose exec api ls -la /data
```

7. Verify end to end through the browser — screen a fraudulent charge in the UI

```
open http://localhost:8501
```

8. Verify the API tier independently

```
curl -s http://127.0.0.1:8000/health

curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

9. Confirm data survives a restart — this is what the volume is for

```
docker compose restart api

sleep 5

curl -s http://127.0.0.1:8000/stats | python3 -m json.tool
```

10. Tear down, keeping the data volume

```
docker compose down

docker volume ls | grep cardguard
```

Test it

Both services start, the UI waits for the API health check, a screening submitted in the browser is stored and visible via /stats, and the data survives a container restart.

Note: Full commands and screenshots are in labs/lab-20-*.md. All transaction data in these labs is generated locally by mockdata.py with a fixed seed. No real cardholder data is used at any point. Use only accounts and data you are authorised to use.

Wrap-Up

Over three days you built and deployed a complete Python application with an AI assistant at your side.

What you built

- A reproducible uv project with locked dependencies.
- An object-oriented domain model for cards, transactions and rules.
- A pandas analytics pipeline that cleans, groups and aggregates transaction data.

- A FastAPI service with typed Pydantic request and response models.
- A Streamlit dashboard and a containerised, verified deployment.

The habits that matter

- Specify before you generate: goal, constraints, inputs, expected output.
- Read every generated line before running it.
- Test the boundaries — thresholds, empty inputs and error paths.
- Keep the analytics layer plain Python so it stays testable.
- Never commit secrets; configuration belongs in environment variables.

Next Steps

- Rebuild CardGuard from scratch on your own machine, using your AI assistant for every step.
- Add a new fraud rule and a matching pytest test, then redeploy the container.
- Swap SQLite for PostgreSQL — the repository layer is the only module that should change.
- Publish your image to a registry and deploy it to a cloud host.
- Practise the four-part prompt pattern on your own work: goal, constraints, inputs, expected output.

Glossary

- **Vibe coding —** Directing an AI coding assistant to produce code that you specify, review, test and own.
- **uv —** A fast Python package and project manager that handles the interpreter, virtual environment and a lockfile.
- **uv.lock —** A lockfile pinning the exact resolved version of every dependency so installs are reproducible.
- **DataFrame —** A labelled two-dimensional table in pandas; a Series is a single column of one.
- **Copy-on-Write —** The pandas 3.0 default under which chained assignment no longer mutates the original frame.
- **split-apply-combine —** The groupby-then-aggregate pattern that answers most questions about a dataset.
- **Encapsulation —** Hiding internal state behind methods and properties so callers depend on behaviour, not layout.
- **Composition —** Modelling a 'has-a' relationship by holding another object, usually preferred over inheritance.
- **Dunder method —** A double-underscore special method such as __init__ or __repr__ that hooks into Python's built-in behaviour.

- **Pydantic —** A library that declares data shapes with type hints and validates them at runtime.
- **FastAPI —** A Python web framework that derives validation and OpenAPI documentation from type annotations.
- **Container image —** A bundle of interpreter, dependencies and application code that runs identically anywhere.
- **Health endpoint —** A lightweight endpoint used to verify that a deployed service is actually running.